
Part I

Fixture Layout Optimization

Chapter 3

Background

In this chapter, we provide background information to clarify terms and concepts that may be unfamiliar to the reader, with the aim of facilitating the understanding of the work presented in this first part of the thesis.

3.1 Constraint Programming

Constraint Programming (CP) is a powerful paradigm for solving combinatorial problems. The basic idea behind CP is that the user specifies the constraints of the problem, while a general-purpose constraint solver is responsible for finding solutions that satisfy them [53].

The three basic concept of a CP model, which are the basic building blocks of an Intelligent Decision Support System (IDSS), are [72]:

- *variables*, representing the decisions to be taken. Each variable is associated with a domain of possible values, where assigning a value corresponds to a specific choice;
- *constraints*, ruling out incompatible choices and defining the relationships and conditions that must be satisfied among variables;
- *parameters*, describing a problem instance and representing fixed input data that influence the formulation and solution of the problem without being decision variables.

A constraint is imposed on a set of variables and restricts the possible combinations of values that these variables can take. For example, consider two variables x and y and the constraint $x < y$. This constraint states that any assignment in which the value of x is smaller than the value of y is valid, whereas any assignment in which the value of x is greater than or equal to the value of y is invalid.

Constraint solvers consider the problem represented in terms of decision variables, parameters and constraints, and find an assignment to all the variables that satisfies the constraints while optimizing the value of an objective function, when specified.

3.2 Mixed Integer Programming

Mixed Integer Programming (MIP) is a mathematical optimization technique in which some decision variables are restricted to be discrete, while others are continuous [78]. The objective function and all constraints are linear, and the goal is to determine the values of the decision variables that optimize the objective while satisfying all constraints. A generic MIP problem can be formulated as:

$$\begin{aligned} \max_x \quad & c^\top x \\ \text{s.t.} \quad & Ax \leq b \\ & x_i \in \mathbb{Z}, \quad \forall i \in I \\ & x_j \in \mathbb{R}, \quad \forall j \notin I \end{aligned}$$

where x is the vector of decision variables, c is the vector of objective coefficients, A is the constraint matrix, b is the vector of constraint bounds, and I denotes the subset of variables constrained to take integer values.

MIP models are solved using constraint solvers capable of handling linear constraints together with integrality restrictions on variables. These solvers differ from those used in Constraint Programming (CP), which focus on propagating constraints over discrete variable domains.

3.3 Particle Swarm Optimization

Particle Swarm Optimization (PSO) is an iterative computational technique inspired by swarm theory and originally motivated by studies of collective behavior in bird flocks [60, 37].

PSO operates on a swarm of particles, each characterized by two vectors: the position p_i and the velocity v_i . Each particle's position represents a candidate solution and is associated with a *fitness* value that measures the solution quality. At each iteration $t = 1, 2, \dots$ the particles explore the search space by updating their velocities and positions according to the following equations:

$$\begin{aligned} v_i^{(t+1)} &= w v_i^{(t)} + c_1 r_1 (p_i^* - p_i^{(t)}) + c_2 r_2 (g^* - p_i^{(t)}) \\ p_i^{(t+1)} &= p_i^{(t)} + v_i^{(t+1)} \end{aligned}$$

where:

- w is the *inertia weight*, controlling how much of the previous velocity is retained;
- c_1 is the *cognitive coefficient*, guiding the particle toward its *personal best position* p_i^* ;
- c_2 is the *social coefficient*, guiding particles toward the *global best position* g^* of the swarm;
- r_1 and r_2 are random numbers typically drawn from a uniform distribution in $[0, 1]$, introducing stochasticity into the cognitive and social components, respectively.

3.4 Principal moments of inertia

To maximize the stability of the workpiece through fixture placement, the *principal moments* of inertia of the fixture layout system can be analyzed.

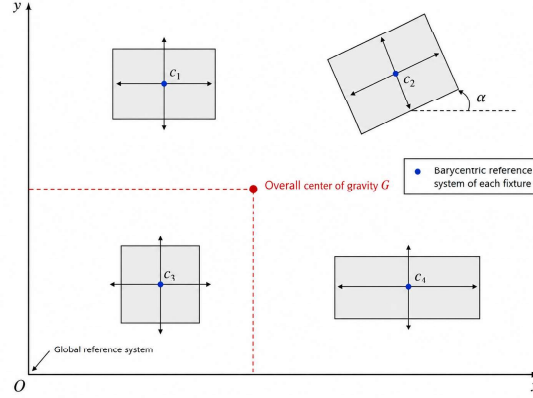


Figure 3.1: Reference system for the computation of the moments of inertia.

The moment of inertia is a geometric property that quantifies a structure's resistance to bending and torsional forces. It is computed from the distribution of area relative to a given axis and plays a crucial role in determining the stiffness of structural elements under load. Higher moments of inertia indicate greater resistance to deformation, making this quantity a key factor in the layout of fixtures that minimize workpiece deflection during machining operations [23, 9].

Figure 3.1 illustrates the reference systems used in the following formulation. A fixed global Cartesian reference system $(O; x, y)$ is defined for the workpiece. Each fixture has its own centroid $c_i = (cx_i, cy_i)$, whereas the point $G = (g_x, g_y)$ denotes the overall centroid of the complete fixture layout.

To compute the principal moments of inertia of a layout composed of n fixtures [5, 14, 19], each fixture i can be represented by a polygon with m vertices $(x_{i,1}, y_{i,1}), \dots, (x_{i,m}, y_{i,m})$, ordered counter-clockwise, expressed directly in the global reference system.

For each fixture i , let A_i denote its area and (cx_i, cy_i) its centroid. Since the centroid is computed from the actual transformed polygon, its coordinates automatically account for any displacement resulting from the physical rotation of the fixture about its rotation center, as shown in Figure 3.1.

The second moments of area of fixture i with respect to the fixed global reference axes are computed directly from its polygon vertices considering the cyclic convention $(x_{i,m+1}, y_{i,m+1}) = (x_{i,1}, y_{i,1})$:

$$\begin{aligned}
J_{x,i}^O &= \frac{1}{12} \sum_{j=1}^m (y_{i,j}^2 + y_{i,j}y_{i,j+1} + y_{i,j+1}^2) (x_{i,j}y_{i,j+1} - x_{i,j+1}y_{i,j}) \\
J_{y,i}^O &= \frac{1}{12} \sum_{j=1}^m (x_{i,j}^2 + x_{i,j}x_{i,j+1} + x_{i,j+1}^2) (x_{i,j}y_{i,j+1} - x_{i,j+1}y_{i,j}) \\
J_{xy,i}^O &= \frac{1}{24} \sum_{j=1}^m (x_{i,j}y_{i,j+1} + 2x_{i,j}y_{i,j} + 2x_{i,j+1}y_{i,j+1} + x_{i,j+1}y_{i,j}) \\
&\quad \cdot (x_{i,j}y_{i,j+1} - x_{i,j+1}y_{i,j})
\end{aligned}$$

Since all fixture polygons are expressed in the same reference system, the second moments of area of the complete fixture layout with respect to the global axes are obtained by direct summation:

$$\begin{aligned}
J_x^O &= \sum_{i=1}^n J_{x,i}^O \\
J_y^O &= \sum_{i=1}^n J_{y,i}^O \\
J_{xy}^O &= \sum_{i=1}^n J_{xy,i}^O
\end{aligned}$$

The total area of the fixture layout is:

$$A = \sum_{i=1}^n A_i$$

and its overall centroid $G = (g_x, g_y)$ is given by

$$g_x = \frac{\sum_{i=1}^n A_i c x_i}{A} \qquad g_y = \frac{\sum_{i=1}^n A_i c y_i}{A}$$

The second moments of area of the complete fixture layout with respect to axes

passing through G and parallel to the global axes are then obtained by applying the parallel-axis theorem to the complete fixture layout:

$$\begin{aligned} J_x &= J_x^O - Ag_y^2 \\ J_y &= J_y^O - Ag_x^2 \\ J_{xy} &= J_{xy}^O - Ag_x g_y \end{aligned}$$

In the above equations, the product of inertia is defined according to the convention

$$J_{xy} = \int_A xy \, dA$$

Finally, the principal moments of inertia are obtained as the eigenvalues of the centroidal second-moment tensor:

$$\begin{aligned} J_\xi &= \frac{J_x + J_y}{2} - \sqrt{\left(\frac{J_x - J_y}{2}\right)^2 + J_{xy}^2} \\ J_\eta &= \frac{J_x + J_y}{2} + \sqrt{\left(\frac{J_x - J_y}{2}\right)^2 + J_{xy}^2} \end{aligned}$$

Layouts that provide good stability for the workpiece are those maximizing the values of J_ξ and J_η .

Chapter 4

Introduction

In wood industry, finished products are typically obtained through machining processes in which a semi-finished component, such as a wood panel, is processed using a machine known as a *work center*, shown in Fig. 4.1a.

A typical work center is equipped with a tool storage system from which the tools required to perform operations such as cutting, milling, and contouring on the panel are automatically selected. However, the use of these tools can induce vibrations and oscillations in the workpiece, thereby reducing the quality of the final product [65, 21]. For this reason, keeping the workpiece as stable and firm as possible during machining is crucial.

To support a workpiece during machining, a work center is equipped with a worktable featuring movable bars that translate along the machine's horizontal axis, above which vacuum suction cup fixtures can be positioned.

The fixtures can be square or rectangular in shape (Fig. 4.1b) and are composed of a square base that allows them to be mounted on the bars and to slide along their length. To improve product quality through an appropriate fixture layout, the fixtures can also be freely rotated to achieve better alignment with the workpiece geometry, with the center of rotation coinciding with the center of the square base.

Determining an appropriate fixture layout for a given workpiece is not always straightforward for a human operator, especially when the workpiece has contoured shapes or includes areas to be extruded. Layouts are often based on the operator's experience; however, such manually designed solutions are not necessarily optimal

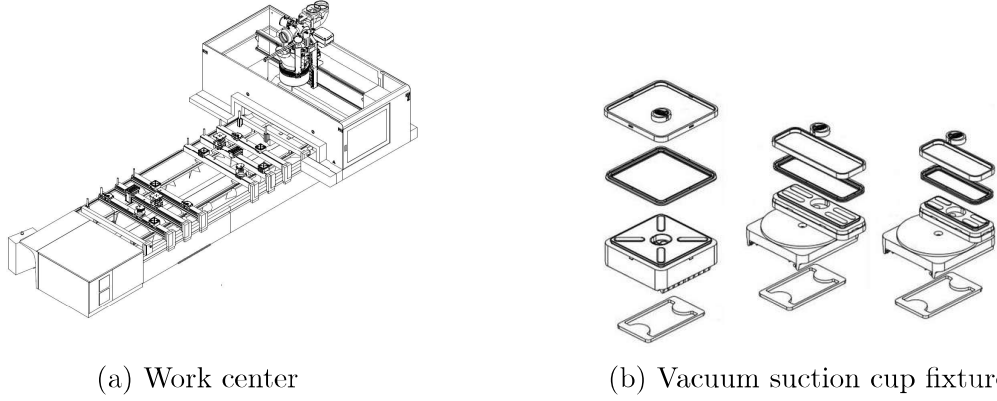


Figure 4.1: On the left, a work center featuring a worktable with movable bars. On the right, fixtures used to secure the workpiece during machining operations.

and can often be improved by considering the structural properties of the fixtures system, such as its resistance to bending and torsional forces.

The purpose of this work is to determine a sound, and possibly optimal, placement of fixtures under the bottom surface of the workpiece, explicitly accounting for the rotation angle of each fixture.

To solve the problem, we first consider a simplified version that neglects fixture rotation and extrusion areas, addressing it through the formulation of a solver-independent Constraint Programming (CP) model. The performance of the model were evaluated using different solvers and two alternative objective functions: one based on the exact computation of the principal moments of inertia, and the other based on the Manhattan distance between pairs of fixtures.

According to the results obtained and considering more real-world test instances, we revisited the objective function and extended the CP formulation to account for extrusion areas. The performance of the extended CP model were then compared with other solution strategies based on a newly formulated Mixed Integer Programming (MIP) model.

The CP model and the MIP model do not consider fixture rotations, as allowing arbitrary rotation angles significantly increases the number of possible configurations and makes the problem substantially harder to solve. Rotations are subsequently incorporated through a refinement stage based on the Particle Swarm Optimization (PSO) meta-heuristic initialized from the layouts obtained from CP

and MIP.

This first part of the thesis is organized as follows. Chapter 5 reviews the state of the art. Chapter 6 introduces the simplified problem formulation and presents the CP model used to solve it. Chapter 7 addresses the complete problem by considering fixture rotations and extrusion areas. Chapter 8 analyzes the integration of the proposed solution strategies. Finally, Chapter 9 concludes this part of the thesis.

Chapter 5

State of the art

The problem of determining a fixture layout that maximizes workpiece stability has been widely studied in the literature. One class of approaches formulates fixture layout as a mathematical optimization problem, often coupled with Finite Element Analysis (FEA) [8] to evaluate workpiece deformation. Menassa and DeVries [40] proposed the use of the Broyden–Fletcher–Goldfarb–Shanno (BFGS) optimization algorithm to determine fixture support locations that minimize deformation. This direction was further explored by Cai et al. [10], who combined nonlinear programming with FEA to optimize fixture layouts under a given force, while Camelio et al. [11] later extended the same approach to minimize shape variations in assembled products. Li and Melkote [33] applied Sequential Quadratic Programming (SQP) to jointly optimize fixture layout and clamping forces. Subsequently, Li et al. [32] proposed a flexible fixture system for sheet metal assembly based on assembly variation analysis using the Lagrangian conditional extremum method, while Zhang et al. [81] developed a nonlinear optimization model solved using the Augmented Lagrangian method.

Evolutionary and meta-heuristic optimization techniques have also been widely investigated. Kulankara et al. [27] proposed a Genetic Algorithm (GA) for the simultaneous optimization of fixture positions and clamping forces. Shortly afterwards, Y. Gene Liao [35] introduced a two-phase GA to identify and refine locating and clamping zones in sheet-metal assembly. Padmanaban et al. [44] combined FEA with discrete and continuous Ant Colony Algorithms (ACA) to re-

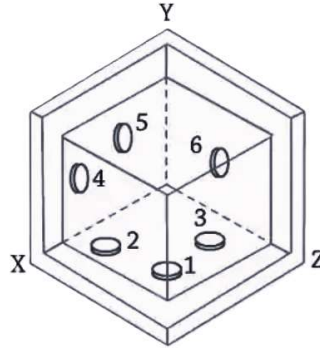


Figure 5.1: Example of the application of the “3-2-1” locating principle for the stabilization of a workpiece

duce sheet metal deformation. Sundararaman et al. [61] proposed a fixture layout optimization approach combining Response Surface Methodology (RSM), Genetic Algorithms (GA), and Particle Swarm Optimization (PSO), reporting superior performance with PSO. More recently, Liu et al. [36] introduced an Improved Particle Swarm Optimization (IPSO) algorithm for the fixture layout optimization of large thin-walled parts.

Learning-based methods have recently emerged as an alternative strategy. Selvakumar et al. [58] combined Artificial Neural Networks (ANNs) with Design of Experiments (DOE) for machining fixture layout optimization. Wang et al. [75] integrated neural networks with the Bat Algorithm (BA) to minimize workpiece deformation. Low et al. [39] proposed a Reinforcement Learning (RL) framework for fixture design, while Wang et al. [74] presented a physics-guided RL approach in which the agent is trained through interaction with an FEA-based simulation.

Although numerous studies on fixture layout optimization exist in the literature, few focus on suction cup fixtures with characteristics similar to those used in wood industry. Most existing research has been conducted in the *metal industry*, where the objective is to determine the number and placement of fixtures according to the “ N -2-1” locating principle in order to minimize workpiece deformation. The “ N -2-1” principle is a generalization of the classical “3-2-1” locating scheme. In the latter, three locators constrain the primary datum plane, two constrain the secondary datum, and one constrains the tertiary datum, thereby fully restricting the six rigid-body degrees of freedom of the workpiece, as shown in Figure 5.1.

The “ N -2-1” principle replaces the three primary locators with N locators, while retaining two and one locators on the secondary and tertiary datum planes [75].

However, this principle cannot be directly applied to work centers in the wood industry, where the workpiece can only be supported from below. Furthermore, the fixtures adopted in the metal industry generally differ from those used in wood industry, as they typically do not allow rotation. While allowing fixture rotation can significantly improve workpiece stability, it also substantially increases the number of possible solutions and therefore the complexity of the optimization problem.

Chapter 6

Simplified problem formulation

The original problem consists of determining, given the workpiece geometry, the available fixtures (possibly of different sizes), and the available worktable bars, a set of fixtures together with their positions, orientations, and types that maximize workpiece stability.

We first consider a simplified version of the problem, assuming that the workpiece does not contain areas to be extruded and neglecting fixture rotations. Under these assumptions, to ensure the feasibility of a layout the following constraints must be satisfied:

1. the number of placed fixtures must not exceed their availability;
2. fixtures must be positioned so as to satisfy the minimum safety distances;
3. fixtures must lie within the boundaries of the workpiece and beneath its surface;
4. fixtures must not overlap each other.

We address this problem by proposing a CP model implemented in the MiniZinc language [43]. The model is parametric with respect to the input data and solver-independent. Consequently, it can be solved using any solver that support MiniZinc, not only CP-based solver, but also alternatives such as Mixed-Integer Programming (MIP) and Constraint Logic Programming (CLP) solvers.

6.1 Model formulation

In this section we describe the CP model in terms of input parameters, variables, constraints, and objective function(s). The model is *parametric* because it is defined in terms of its parameters: changing the input data does not affect the model itself.

6.1.1 Parameters

The input parameters shaping a particular problem instance are:

- the width **WTab** and height **HTab** of the machine worktable;
- the minimum horizontal distance **WMin** between two fixtures placed on different bars;
- the number **NTypes** of different fixture types;
- the number **NFix_i** of available fixtures for each type $i \in [1..NTypes]$;
- the total number of available fixtures $NFix = \sum_{i=1}^{NTypes} NFix_i$;
- the minimum **NMin** and maximum **NMax** number of fixtures to place;
- the width **WFix_i** and height **HFix_i** of each fixture of type i ;
- the workpiece shape, defined by a set of linear inequalities of the form $ax + by + c \leq 0$;

6.1.2 Variables

To determine which fixtures to use and their positions on the worktable, we define three arrays of variables x , y , t of size **NFix** such that, for $i = 1, \dots, NFix$:

- (x_i, y_i) is the coordinate of the lower-left corner of the i -th fixture, with $x_i \in [0..WTab]$ and $y_i \in [0..HTab]$
- t_i is the type of the i -th fixture, with $t_i \in [0..NTypes]$

We also define a variable $n_i \in [0..\text{NFix}_{t_i}]$ for each $i \in [0..\text{NTypes}]$ counting the number of fixtures of type i , and a variable n corresponding to the total number of fixtures employed.

6.1.3 Constraints

For each fixture of type $i \in [1..\text{NTypes}]$, we impose the CP *global* constraint [4]:

$$n_i = \text{count}(t, i)$$

counting the number of fixtures in t with type i , and set $n = \sum_i n_i$.

To break symmetries that arise when fixtures of the same type are permuted without changing their spatial arrangement, we impose a lexicographic ordering such that $(x_{i-1}, y_{i-1}) \preceq (x_i, y_i)$ for $i \in [2..\text{NFix}]$ through the `lex_chain` global constraint.

Because we may not use all the available fixtures, we enforce the invariant:

$$(i > n) \iff (x_i = \text{WTab} \wedge y_i = \text{HTab} \wedge t_i = 0)$$

The workpiece region is defined by a set of linear inequalities of the form $\mathbf{a}x + \mathbf{b}y + \mathbf{c} \leq 0$. Therefore, to ensure that the fixtures lie within the workpiece boundaries, each fixture coordinate (x_i, y_i) , for $i \leq n$, must satisfy the condition $\mathbf{a}x' + \mathbf{b}y' + \mathbf{c} \leq 0$ where:

$$x'_i = \begin{cases} x_i & \text{if } a \leq 0 \\ x_i + \text{WFix}_{t_i} & \text{if } a > 0 \end{cases} \quad y'_i = \begin{cases} y_i & \text{if } b \leq 0 \\ y_i + \text{HFix}_{t_i} & \text{if } b > 0 \end{cases}$$

Indeed, according to the sign of coefficients $\mathbf{a}$ and $\mathbf{b}$, we only have to check that one vertex of the rectangular fixture is within the workpiece region. For example, if the inequality is $3x - 2y + 4 \leq 0$, we impose $3(x_i + \text{WFix}_{t_i}) - 2y_i + 4 \leq 0$ to enforce the lower-right corner within the hyperplane defined by that inequality. If so, also all the other fixture points must be within that region, as shown in Figure 6.1.

To encode the no-overlap constraint between fixtures we employ the `diffn`

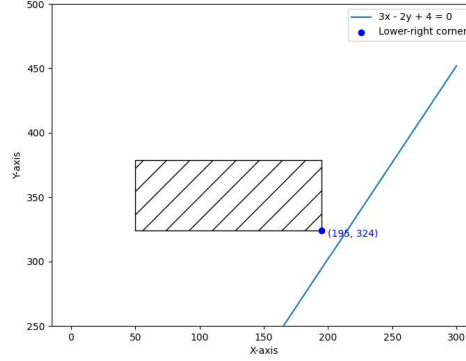


Figure 6.1: Geometric constraint for fixtures placement

global constraint [4]:

$$\text{diffn}(x, y, [\mathbf{WFix}_{t_i} \mid i \in [1..n]], [\mathbf{HFix}_{t_i} \mid i \in [1..n]])$$

This constraint ensures that each rectangle with origin in the lower-left corener (x_i, y_i) , width $\mathbf{WFix}_{t_i}$ and height $\mathbf{HFix}_{t_i}$ is not overlapping. Specifically, we use the *non-strict* version of **diffn**, in which zero-width rectangles can be positioned anywhere, ignoring the $\mathbf{NFix} - n$ unemployed fixtures.

The same constraint can be naturally extended to prevent fixtures from being placed within areas to be extruded, as will be described in Section 7.2.2.

Finally, we guarantee that the horizontal distance between two fixtures placed on different bars is at least $\mathbf{WMin}$. Formally, for each $i \in [1..n - 1]$ we impose that:

$$x_i \neq x_{i+1} \implies x_i + \mathbf{WFix}_{t_i} + \mathbf{WMin} \leq x_{i+1}$$

6.1.4 Objective function

As illustrated in Chapter 3, is possible to maximize workpiece stability by maximizing the principal moments of inertia. Consequently, the ideal objective function would be:

$$f_{iner} = J_{\xi} + J_{\eta} \tag{6.1}$$

where J_ξ and J_η are the principal moments of inertia defined in Section 3.4.

However, computing J_ξ and J_η involves nonlinear operations such as squaring, square roots, and transcendental functions (sine and cosine), which are difficult to handle efficiently with CP solvers [31].

An analysis of the moment of inertia for different fixture layouts, together with previous studies on workpiece stability [24, 54, 40, 41, 73], indicates that workpiece stability increases when fixtures are placed far apart from one another and close to the workpiece perimeter. Based on these observations, we devised an alternative objective function, $f_{dist}^{semi-perimeter}$, that maximizes the sum of the pairwise Manhattan distances between fixture centroids, augmented by the semi-perimeter of the selected fixtures.

$$f_{dist}^{semi-perimeter} = \sum_{1 \leq i < j \leq n} (|cx_i - cx_j| + |cy_i - cy_j|) + \sum_{k=1}^n (\text{WFix}_{t_k} + \text{HFix}_{t_k}) \quad (6.2)$$

where $(cx_i, cy_i) = (x_i + \frac{\text{WFix}_{t_i}}{2}, y_i + \frac{\text{HFix}_{t_i}}{2})$ are the coordinates of the centroid of the i -th placed fixture, having type t_i .

6.2 Case study

We evaluated our approach on a real case where the workpiece is a stair step of a spiral staircase. The workpiece area, shown in Figure. 6.3, is defined by the following inequalities:

$$\begin{aligned} (a) : x &\geq 20 & (d) : x &\leq 689 \\ (b) : y &\leq 0.20x + 257.91 & (e) : y &\geq 0.54x - 356.21 \\ (c) : y &\leq -0.58x + 781.92 & (f) : y &\geq -0.20x + 139.06 \end{aligned}$$

The width and height of the worktable are respectively $\text{WTab} = 700$ and $\text{HTab} = 400$. The minimal horizontal distance between fixtures is $\text{WMin} = 140$. We have two different types of fixtures available:

- five square fixtures of dimension 145×145 mm;

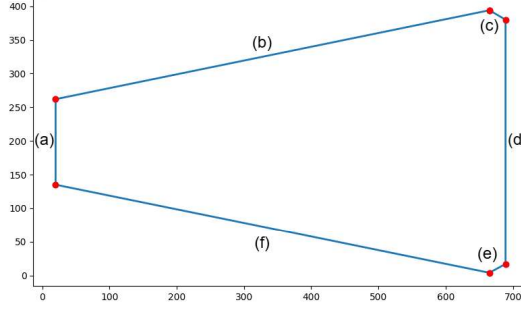


Figure 6.2: Stair step of a spiral staircase

Figure 6.3: A polygonal workpiece representing a stair step

- five rectangular fixtures of dimension 145×55 mm.

So, $N\text{Fix}_1 = N\text{Fix}_2 = 5$ with $W\text{Fix}_1 = H\text{Fix}_1 = W\text{Fix}_2 = 145$ and $H\text{Fix}_2 = 55$.

We set an upper bound $N\text{Max}$ on the number of fixtures (i.e., we impose $n \leq N\text{Max}$) and we evaluated different values of $N\text{Max}$ in the range $[2..10]$. We assume a minimum number of 2 fixtures (i.e., $n \geq 2$) as placing only one fixture is not a realistic case.

We defined two models, one using the objective function f_{iner} and the other using $f_{dist}^{semi-perimeter}$. Both were implemented in Minizinc [43]. For the model using $f_{dist}^{semi-perimeter}$, four solvers were evaluated: Chuffed [13], Gecode [56], OR-Tools [17] and Gurobi [18]. For the f_{iner} model, we restricted our evaluation to solvers that support floating-point variables, namely Gecode and Gurobi.

All experiments were conducted with a time limit of 300 seconds on a 64-bit Windows machine equipped with an Intel Core i5 CPU and 16 GB of RAM.

Table 6.1 and Table 6.2 report the results obtained using f_{iner} and $f_{dist}^{semi-perimeter}$ as objective functions, respectively, across different values of $N\text{Max}$. Specifically, Table 6.2 doesn't report the value of $f_{dist}^{semi-perimeter}$ but the corresponding *a posteriori* values of f_{iner} for the best solutions found by the solvers.

In Table 6.1 the result for the Gecode solver are not reported, as it fails to find any feasible solution within the time limit. On the other hand, Gurobi being a MIP solver that handle continuous variables, is able to find optimal solutions for

$N\text{Max} \leq 5$ but is not able to find any solution for $N\text{Max} \geq 6$.

When using $f_{dist}^{semi-perimeter}$ as the objective function, Table 6.2 shows that all solvers are able to find at least one feasible solution for every value of $N\text{Max}$, with Chuffed and Gurobi solving all instances to optimality. Overall, the best performance is achieved by Chuffed, which is able to reach the optimal solutions in less time than Gurobi.

In the tables the optimal solutions found are reported in bold, the $f_{dist}^{semi-perimeter}$ objective function can admit multiple equivalent optimal solutions; however, these solutions may lead to different values of f_{iner} , due to the high sensitivity of the moments of inertia to small fixture displacements, as reported for $N\text{Max} \in 3, 4, 5$ in Table 6.2.

Comparing the two tables, we can observe that solutions generated using $f_{dist}^{semi-perimeter}$ generally achieve lower f_{iner} values, but can be computed significantly faster, making this objective function particularly suitable for large workpieces or instances requiring a high number of fixtures. Indeed, comparing the runtimes obtained by Gurobi with the two objective functions shows that solving the problem with $f_{dist}^{semi-perimeter}$ is on average more than 150 times faster than using f_{iner} .

Table 6.1: Results for f_{iner} . All f_{iner} values reported in units of 10^9 . Bold fold indicates the best f_{iner} value found before the timeout.

		NMax								
Solver	Metric	2	3	4	5	6	7	8	9	10
Gurobi	Runtime	14250	189000	90000	258000	-	-	-	-	-
	f_{iner} [ms]	2.65072	3.79990	3.79990	3.87506	3.89034	-	-	-	-

Table 6.2: Results using $f_{dist}^{semi-perimeter}$. Bold fold denotes the f_{iner} , reported in units of 10^9 , corresponding to a $f_{dist}^{semi-perimeter}$ -optimal solution.

Solver	Metric	NMax								
		2	3	4	5	6	7	8	9	10
Chuffed	Runtime	1566	904	2039	3333	3700	2533	4873	5222	5593
	[ms] f_{iner}	2.64416	2.38480	2.58584	3.54968	3.80905	3.80905	3.80905	3.80905	3.80905
Gurobi	Runtime	503	745	961	1098	1540	1856	4674	5534	10840
	[ms] f_{iner}	2.64416	2.38350	2.58584	3.08459	3.80905	3.80905	3.80905	3.80905	3.80905
Gecode	Runtime	549	18292	–	–	–	–	–	–	–
	[ms] f_{iner}	2.64416	2.39170	2.58584	2.58584	2.58584	3.73308	2.58584	2.58584	2.58584
OR-Tools	Runtime	882	7804	260000	–	–	–	–	–	–
	[ms] f_{iner}	2.64416	2.16912	2.24606	3.34565	3.34565	3.38321	3.73467	3.73467	3.67863

Tables 6.3 and 6.4 present the visual results obtained from the different solvers using the $f_{dist}^{semi-perimeter}$ objective function, evaluated across varying values of **NMax**, while Tables 6.5 and 6.6 report the corresponding layouts obtained with Gurobi when optimizing f_{iner} .

The layouts obtained with $f_{dist}^{semi-perimeter}$ are generally similar across solvers. However, when the number of fixtures is limited to three, Chuffed, Gecode, and OR-Tools tend to place a smaller fixture on the left side of the workpiece, below its center, whereas Gurobi positions the smaller fixture closer to the center and places two larger fixtures on the right side, leading to a higher f_{iner} value, as reported in Table 6.2.

As expected, optimizing f_{iner} promotes the selection of larger fixtures, since larger support areas result in higher moments of inertia. For instance, with three fixtures, the solution provided by Gurobi using the f_{iner} objective function selects only the largest available fixtures. This configuration is maintained also for **NMax** = 4, where the solver avoids introducing an additional fixture.

Table 6.3: Visual results for solvers' solutions using $f_{dist}^{semi-perimeter}$ as objective function ($NMax = 2-4$).

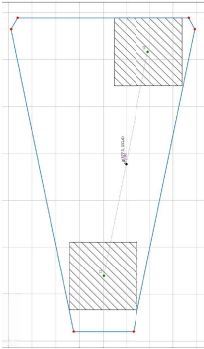
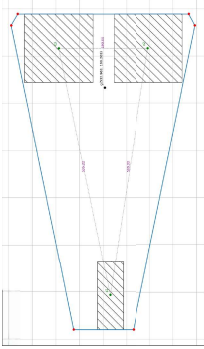
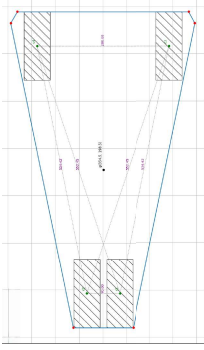
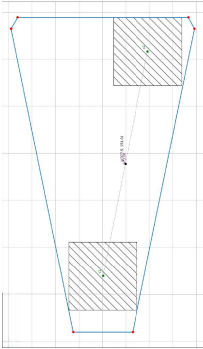
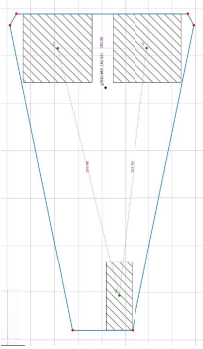
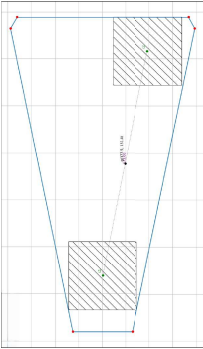
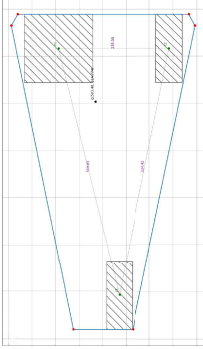
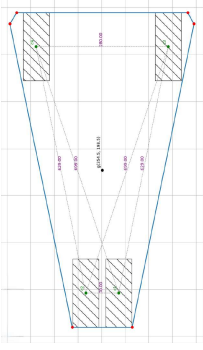
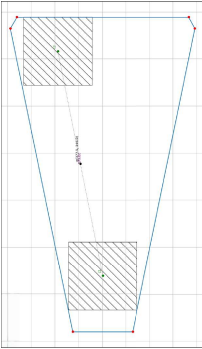
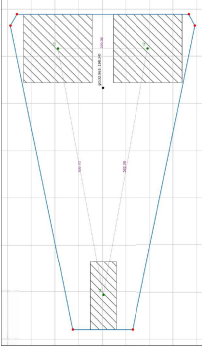
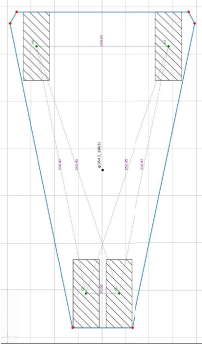
Solver	NMax			
	2	3	4	
Chuffed				
Gecode ($f_{dist}^{semi-perimeter}$)				
OR-Tools				
Gurobi ($f_{dist}^{semi-perimeter}$)				

Table 6.4: Visual results for solvers’ solutions using $f_{dist}^{semi-perimeter}$ objective function (NMax = 5–10).

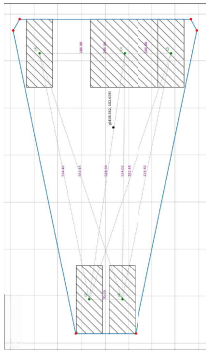
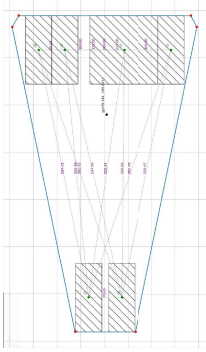
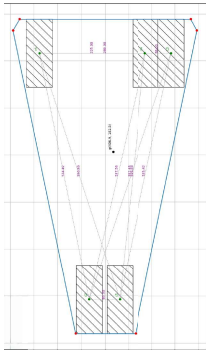
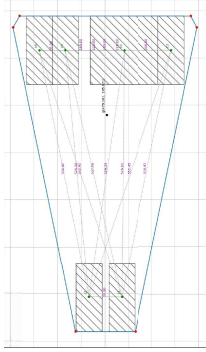
Solver	NMax	
	5	6–10
Chuffed		
OR-Tools		
Gurobi ($f_{dist}^{semi-perimeter}$)		

Table 6.5: Visual results for Gurobi solver solutions using f_{iner} objective function (NMax = 2–4).

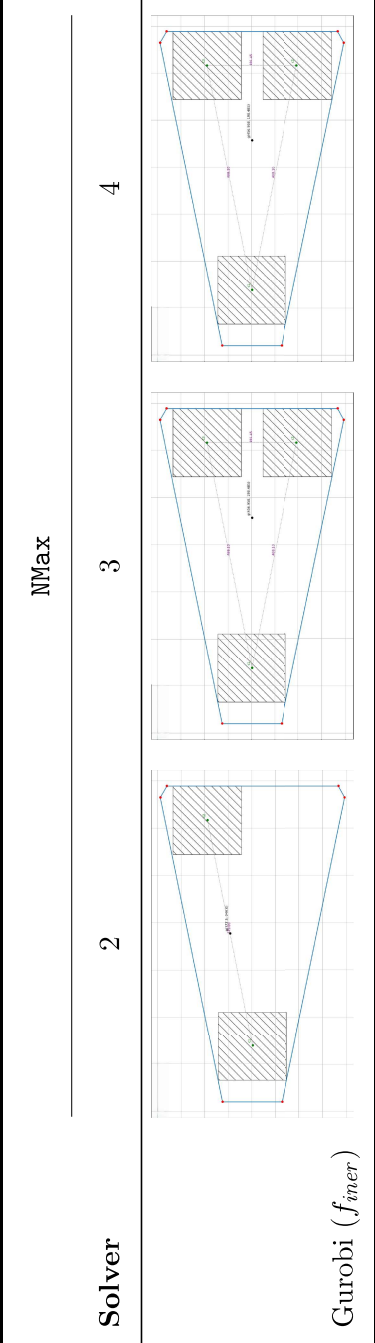
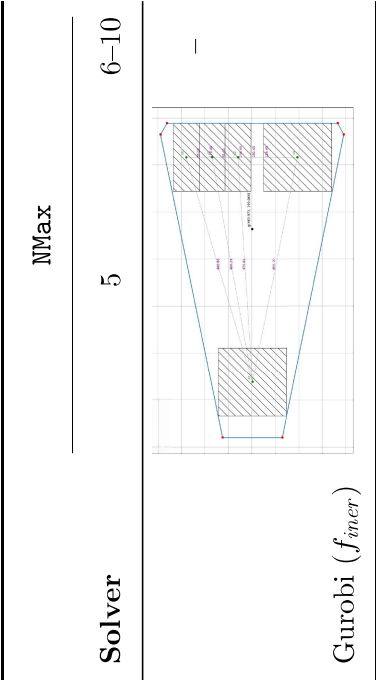


Table 6.6: Visual results for Gurobi solver solutions using f_{iner} objective function (NMax = 5–10).



Chapter 7

Considering fixture rotations and extrusion areas

Having solved the simplified version of the problem, where fixture rotations and areas to be extruded are not considered, using the CP model described in Chapter 6, we now address the complete problem by incorporating both fixture rotations and the presence of areas to be extruded within the workpiece.

The constraints remain the same as those described previously, with the additional requirement that fixtures must not be positioned beneath areas to be extruded. Therefore, the resulting constraints that need to be satisfied are:

1. the number of placed fixtures must not exceed their availability;
2. fixtures must satisfy the minimum safety distances;
3. fixtures must lie within the boundaries of the workpiece and beneath its surface;
4. fixtures must not overlap;
5. fixtures must not be located beneath areas to be extruded.

To clarify layout feasibility, Figure 7.1 illustrates examples of valid and invalid fixture placements. In the figure, fixture F_6 is invalid because it violates the minimum horizontal safety distance with respect to fixtures F_3 and F_4 . Fixture

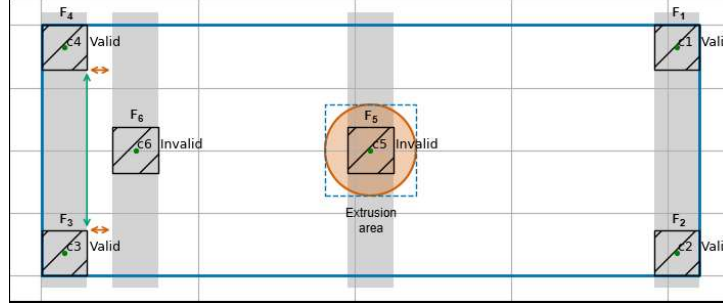


Figure 7.1: Example of valid and invalid fixture placements. For a fixture placement to be valid, the fixture must lie within the workpiece boundaries, satisfy the required safety distances from the other fixtures, and not be positioned beneath an area to be extruded.

F_5 , on the other hand, is invalid because it is positioned beneath an area to be extruded.

To address the complete problem, we extend the CP model introduced in Section 6.1 to handle the new constraint. We also formulate a new Mixed Integer Programming (MIP) model and compare the performance of the two formulations on real-world test cases.

Both the CP and MIP models can be solved using appropriate solvers to obtain feasible layout. However, neither approach accounts for fixture rotations, since allowing rotation angles other than multiples of 90° significantly increases the number of possible configurations and makes the problem substantially harder to solve.

Moreover, inclined fixtures complicate overlap checking and, in CP solvers, require geometric approximations due to the lack of native support for continuous variables. For these reasons, fixture rotations are handled in a final refinement stage based on a Particle Swarm Optimization (PSO) meta-heuristic, which iteratively improves the initial solution, provided by the different approaches.

The introduction of rotation at this stage provides additional flexibility in fixture placement, enlarging the set of feasible placements and allowing fixture centroids to be positioned closer to workpiece boundaries or areas to be extruded.

7.1 Parameters and objective function

The problem parameters extend those introduced in Section 6.1.1 with information related to the areas to be extruded.

All the proposed solution strategies are defined over the same set of input parameters, listed below:

- the width **WTab** and height **HTab** of the machine worktable;
- the minimum horizontal distance **WMin** between two fixtures placed on different bars;
- the minimum vertical distance **HMin** between two fixtures placed on the same bar;
- the number **NTypes** of different fixture types;
- the number **NFix_i** of available fixtures for each type $i \in [1..NTypes]$;
- the total number of available fixtures $NFix = \sum_{i=1}^{NTypes} NFix_i$;
- the minimum **NMin** and maximum **NMax** number of fixtures to place;
- the width **WFix_i** and height **HFix_i** of each fixture of type i ;
- the workpiece shape, by a set of linear inequalities of the form $\mathbf{a}x + \mathbf{b}y + \mathbf{c} \leq 0$;
- the number **NExt** of areas to be extruded;
- the width **WExt_i**, height **HExt_i**, and the coordinates (**XExt_i**, **YExt_i**) of the lower-left corner of the smallest axis-aligned rectangle enclosing each area $i \in [1..NExt]$ to be extruded.

To facilitate the solving process, areas to be extruded are approximated by their minimum bounding rectangles. In practice, these areas typically correspond to simple geometries, such as circles, rectangles, or ellipses, while more complex shapes are usually refined manually.

Compared with the simplified formulation, we also introduce in the parameters the minimum vertical distance between fixtures. This parameter was previously

omitted to allow the placement of multiple fixtures beneath the workpiece and to facilitate the comparison between $f_{dist}^{semi-perimeter}$ and f_{iner} for increasing values of **NMax** (see Section 6.2).

To evaluate the quality of a fixture layout, two objective functions are considered f_{dist}^{area} and f_{iner} .

The CP and MIP models optimize the objective function f_{dist}^{area} , which maximizes the sum of the pairwise Manhattan distances between fixture centroids, augmented by the total area of the selected fixtures:

$$f_{dist}^{area} = \sum_{1 \leq i < j \leq n} (|cx_i - cx_j| + |cy_i - cy_j|) + \sum_{k=1}^n (W\mathbf{Fix}_{t_k} \cdot H\mathbf{Fix}_{t_k}) \quad (7.1)$$

We adopt the area term instead of the semi-perimeter to prioritize fixtures with larger contact surfaces. We empirically verified that this approach provides a stronger surrogate for the inertia objective in more complex geometries. In the experiments of Section 7.5, the Pearson correlation coefficient [6] between f_{dist}^{area} and f_{iner} when using the area-based formulation is higher (0.79) than the semi-perimeter-based one (0.39).

Unlike the CP and MIP models, PSO is not affected by the nonlinear operations required to evaluate f_{iner} . Consequently, it does not rely on the alternative objective function; instead, it directly incorporates the moments of inertia into the optimization algorithm.

7.2 CP model

We formulate the model by extending the CP formulation presented in Section 6.1 through a revised definition of the decision variables and a reformulation of some constraints, which will be presented in the following sections.

7.2.1 Variables

The fixture placement and type are represented by three arrays of integer decision variables, x , y , and t , for $i = 1, \dots, \mathbf{NMax}$:

- $(x_i, y_i) \in [0..\text{WTab}] \times [0..\text{HTab}]$ denotes the coordinates of the lower-left corner of the i -th fixture;
- $t_i \in [0..\text{NTypes}]$ denotes the type of the i -th fixture, where type 0 represents an unused fixture.

For each fixture type $i \in [1..\text{NTypes}]$, an integer variable $n_i \in [0..\text{NFix}_i]$ denotes the number of selected fixtures of type i , and a variable $n \in [\text{NMin}..\text{NMax}]$ represents the total number of selected fixtures.

Compared with the formulation presented in 6.1, the arrays x and y and the variable n are now bounded by **NMax**, rather than by **NFix**, since the maximum number of fixtures that can be placed is known for each workpiece and $\text{NMax} \leq \text{NFix}$ this tighter bound reduces the search space.

7.2.2 Constraints

For each fixture type $i \in [1..\text{NTypes}]$, the **count** global constraint is used to compute the number of occurrences of value i in the array t :

$$n_i = \text{count}(t, i)$$

The total number of selected fixtures is then defined as

$$n = \sum_{i=1}^{\text{NTypes}} n_i$$

Fixture availability is enforced by requiring that the number of selected fixtures of each type does not exceed the number available:

$$n_i \leq \text{NFix}_i, \quad \forall i \in [1..\text{NTypes}]$$

Since not all available fixtures must necessarily be used, we enforce the following invariant:

$$(i > n) \iff (x_i = \text{WTab} \wedge y_i = \text{HTab} \wedge t_i = 0), \quad \forall i \in [1..\text{NMax}]$$

This invariant ensures that all unused fixtures are assigned the dummy type

0 and placed at the dummy location (`WTab`, `HTab`), which lies outside the feasible region.

We guarantee that exactly $\text{NMax} - n$ fixtures remain unused by imposing

$$\text{count}(t, 0) = \text{NMax} - n$$

The workpiece shape is defined by linear inequalities of the form $\mathbf{a}x + \mathbf{b}y + \mathbf{c} \leq 0$. To ensure that each selected fixture i is placed within the workpiece, we must satisfy $\mathbf{a}x'_i + \mathbf{b}y'_i + \mathbf{c} \leq 0$, where x'_i and y'_i are defined as in Section 6.1.3.

Areas to be extruded are approximated by axis-aligned rectangles. The non-overlap between these areas and the fixtures is enforced using the `diffn` global constraint [4]:

$$\text{diffn}(x \oplus \text{XExt}, y \oplus \text{YExt}, [\text{WFix}_{t_i} \mid i \in [1..n]] \oplus \text{WExt}, [\text{HFix}_{t_i} \mid i \in [1..n]] \oplus \text{HExt})$$

where $\oplus$ denotes array concatenation. Specifically, we use the *non-strict* version of `diffn`, ignoring the unused fixtures.

To enforce the minimum *vertical* distance `HMin` between fixtures placed on the same bar, and the minimum *horizontal* distance `WMin` between fixtures placed on different bars we impose:

$$\forall 1 \leq i < j \leq n : \begin{cases} y_i + \text{HFix}_{t_i} + \text{HMin} \leq y_j \vee y_j + \text{HFix}_{t_j} + \text{HMin} \leq y_i, & \text{if } cx_i = cx_j, \\ cx_i + \text{WFix}_{t_i} + \text{WMin} \leq cx_j, & \text{if } cx_i \neq cx_j. \end{cases}$$

where cx_i denotes the x -coordinate of the centroid of fixture i . Centroid coordinates are used instead of lower-left coordinates because fixtures may be wider than their supporting bar: while x_i may fall outside the horizontal span of the bar, the centroid cx_i always lies within it.

To compute the fixture centroid coordinates while preserving integer-valued variables, the coordinates are obtained using integer division of the fixture dimensions. For each fixture $i \in [1..\text{NMax}]$, we define:

$$cx_i = x_i + \left\lfloor \frac{\text{WFix}_{t_i}}{2} \right\rfloor \quad cy_i = y_i + \left\lfloor \frac{\text{HFix}_{t_i}}{2} \right\rfloor$$

To improve the propagation of the integer division, the following redundant linear constraints were introduced:

$$\begin{aligned} 2 \cdot cx_i - 2 \cdot x_i &= \text{WFix}_{t_i} - (\text{WFix}_{t_i} \bmod 2) \\ 2 \cdot cy_i - 2 \cdot y_i &= \text{HFix}_{t_i} - (\text{HFix}_{t_i} \bmod 2) \end{aligned}$$

Finally, the model exhibits symmetries whenever fixtures of the same type are permuted without changing their spatial arrangement. To eliminate these equivalent solutions, a lexicographic ordering is imposed on the fixture positions using the `lex_chain` global constraint:

$$(x_{i-1}, y_{i-1}) \preceq (x_i, y_i) \quad i \in [2..\text{NMax}]$$

7.3 MIP model

In this section, we present the Mixed-Integer Programming (MIP) formulation for the fixture layout optimization problem. Compared to the CP model described in Section 7.2, all the constraints are expressed in linear form, and the positional decision variables are continuous rather than discrete.

This important relaxation comes from the nature of MIP solving which, unlike most CP solvers, naturally handles non-negative continuous variables. Nevertheless, a number of discrete variables are still required to capture the combinatorial structure of the problem, as detailed below.

7.3.1 Variables

Variables x_i and y_i for $i = 1, \dots, \text{NMax}$ have exactly the same semantics as those defined in Section 7.2.1, with the only difference that they are now continuous

rather than discrete.

Fixture types are encoded with a 0-1 matrix T of size $\text{NMax} \times \text{NTypes}$, whose semantics is:

$$T_{i,j} = \begin{cases} 1 & \text{if fixture } i \text{ is selected and has type } j, \\ 0 & \text{otherwise.} \end{cases}$$

The linearization of the constraints comes at the expense of the introduction of auxiliary 0-1 variables, detailed as follows:

- $used_i$, denoting whether fixture i is selected;
- $same_bar_{i,j}$, denoting whether fixtures i and j lie on the same bar;
- $above_in_bar_{i,j}$, $below_in_bar_{i,j}$, denoting whether fixture i is above or below fixture j on the same bar;
- $above_{i,j}$, $below_{i,j}$, $left_{i,j}$, $right_{i,j}$, indicating whether fixture i is positioned above, below, to the left, or to the right of fixture j , respectively.
- $above_ext_{i,j}$, $below_ext_{i,j}$, $left_ext_{i,j}$, $right_ext_{i,j}$ indicating whether fixture i is positioned above, below, to the left, or to the right of an area j to be extruded, respectively.

We also define auxiliary variables w_i , h_i and t_i to denote, respectively, the width w_i , the height h_i , and the type t_i of each fixture.

7.3.2 Constraints

For each fixture $i \in [1..\text{NMax}]$ we impose:

$$\begin{aligned} used_i &= \sum_{j=1}^{\text{NTypes}} T_{i,j} & w_i &= \sum_{j=1}^{\text{NTypes}} \text{WFix}_j \cdot T_{i,j} \\ h_i &= \sum_{j=1}^{\text{NTypes}} \text{HFix}_j \cdot T_{i,j} & t_i &= \sum_{j=1}^{\text{NTypes}} j \cdot T_{i,j} \end{aligned}$$

These equations ensure that the width, height, and type assigned to each fixture are consistent and that unused fixtures have dimensions and type equal to 0.

To enforce availability constraints, we impose:

$$\sum_{i=1}^{\text{NMax}} T_{i,j} \leq \text{NFix}_j, \quad \forall j \in [1..\text{NTypes}] \quad \text{NMin} \leq \sum_{i=1}^{\text{NMax}} \text{used}_i \leq \text{NMax}$$

To determine whether two fixtures lie on the same bar for all $i \in [1..\text{NMax}]$ and $j \in [i + 1..\text{NMax}]$ we define the following constraints:

$$\begin{aligned} \text{same_bar}_{i,j} &\leq \text{used}_i & \text{same_bar}_{i,j} &\leq \text{used}_j \\ \text{same_bar}_{i,j} &= \text{above_in_bar}_{i,j} + \text{below_in_bar}_{i,j} \\ cx_i - cx_j &\leq M(1 - \text{same_bar}_{i,j}) & cx_j - cx_i &\leq M(1 - \text{same_bar}_{i,j}) \end{aligned}$$

where $M = \text{WTab} \cdot \text{HTab}$ is the “big- M ” constant. To enforce the appropriate safety distances:

$$\begin{aligned} cx_i + \text{WMin} &\leq cx_j + M(\text{same_bar}_{i,j} + \text{right}_{i,j}) \\ cx_j + \text{WMin} &\leq cx_i + M(\text{same_bar}_{i,j} + \text{left}_{i,j}) \\ y_i + h_i + \text{HMin} - y_j &\leq M(1 - \text{above_in_bar}_{i,j}) \\ y_j + h_j + \text{HMin} - y_i &\leq M(1 - \text{below_in_bar}_{i,j}) \end{aligned}$$

To prevent overlap between fixtures, for all $i, j \in [1..\text{NMax}]$ with $i < j$, we impose:

$$\begin{aligned} \text{above}_{i,j} + \text{below}_{i,j} + \text{left}_{i,j} + \text{right}_{i,j} &\geq 1 \\ x_i + w_i &\leq x_j + M(1 - \text{left}_{i,j}) & y_i + h_i &\leq y_j + M(1 - \text{below}_{i,j}) \\ x_j + w_j &\leq x_i + M(1 - \text{right}_{i,j}) & y_j + h_j &\leq y_i + M(1 - \text{above}_{i,j}) \end{aligned}$$

The same constraints are applied to prevent overlap between fixtures and areas to be extruded. For each fixture $i \in [1..\text{NMax}]$ and extrusion area $j \in [1..\text{NExt}]$, the above inequalities are instantiated using the variables $\text{above_ext}_{i,j}$, $\text{below_ext}_{i,j}$, $\text{left_ext}_{i,j}$, and $\text{right_ext}_{i,j}$.

The workpiece is defined by linear inequalities of the form $\mathbf{a}x_i + \mathbf{b}y_i + \mathbf{c} \leq 0$,

so for each fixture $i \in [1..\mathbf{NMax}]$ we impose $\mathbf{a}x'_i + \mathbf{b}y'_i + \mathbf{c} \leq 0$ where x'_i and y'_i are defined as in Section 6.1.3.

To break symmetries, we impose an ordering on fixture types: $t_i \geq t_{i+1}$ for $i \in [1..\mathbf{NMax} - 1]$.

We linearize f_{dist}^{area} with a big-M formulation unfolding each term of the form $u = |v|$ into:

$$u \geq v \wedge u \geq -v \wedge u \leq v + M(1 - b) \wedge u \leq -v + Mb \wedge b \in \{0, 1\}.$$

7.4 Particle Swarm Optimization algorithm

Particle Swarm Optimization (PSO) is employed to explicitly account for fixture rotation, considering as objective function $f_{iner} = J_\xi + J_\eta$.

PSO is particularly well suited to this refinement phase due to its fast convergence and strong global search capability in high-dimensional and multi-objective optimization problems. Additionally, its simple structure, limited number of control parameters, and ease of hybridization enable robust optimization across diverse manufacturing, scheduling, and design applications when compared with Genetic Algorithms (GA), Simulated Annealing (SA), and gradient-based methods [28].

In our formulation, the position of each particle $s \in [1, \mathbf{swarm_size}]$ is represented by a matrix P_s with four rows and $\mathbf{NMax}$ columns. Each column of P_s represents a fixture, while each row corresponds to one of the decision variables:

- $P_s[1, j]$: the x -coordinate of the lower-left corner of the j -th fixture;
- $P_s[2, j]$: the y -coordinate of the lower-left corner of the j -th fixture;
- $P_s[3, j]$: the type t associated with the j -th fixture;
- $P_s[4, j]$: the rotation angle α of the j -th fixture.

Since PSO operates on continuous variable values, during the execution the fixture type values in $P_s[3, j]$ are rounded to the nearest integer to ensure valid fixture types and evaluate the corresponding constraints.

Each position matrix P_s represents a candidate solution associated with particle s . The availability of an initial solution obtained from the CP model and the MIP model ensures that the swarm is initialized from feasible layouts.

Algorithm 1 presents the pseudocode of the implemented PSO approach. Given the swarm size `swarm_size`, the maximum number of iterations `max_iter`, the inertia weight w , the cognitive coefficient c_1 , the social coefficient c_2 , the variable bounds, the initial feasible layout P_0 , and the maximum number of fixtures `NMax`, the algorithm returns the optimized feasible fixture layout P^* .

During the initialization phase (lines 1–5 of Algorithm 1), the first particle is assigned with the provided initial solution P_0 , whereas the remaining particles are initialized using randomly perturbed versions of P_0 . The velocity matrix V_s of each particle s , which stores the velocity components corresponding to each element of the position matrix P_s , is randomly initialized, and the personal best position P_s^* is initially set equal to the particle's starting position.

In this formulation, the linear constraints of the MIP model are incorporated into the PSO framework and treated as soft constraints, with the non-overlap constraints adapted to account for fixture rotations. Accordingly, for each particle position P_s , the violation of the j -th constraint is evaluated by means of a binary function $cv_j(P_s)$, which indicates whether the constraint is violated or satisfied. The overall constraint violation $cv(P_s)$ for position P_s is computed as the sum of each individual constraint violation.

Once all particles have been initialized, they are evaluated in terms of the objective function $f_{iner}(P_s)$ and the constraint violation $cv(P_s)$, as reported in lines 7–11 of Algorithm 1. Based on this evaluation, the global best position of the swarm, denoted by P^* , is identified.

When fixtures are axis-aligned, non-overlap can be enforced through coordinate-wise comparisons by exploiting the orientation of the rectangle edges. Introducing fixture rotations invalidates this criterion, since the global position of each vertex depends on the rotation angle and must be computed through trigonometric transformations. Consequently, non-overlap verification becomes orientation-dependent, increasing the implementation complexity. Furthermore, vertex-based checks are no longer sufficient for oriented rectangles, as two fixtures may intersect even when no vertex of either rectangle lies inside the other, for example in

Algorithm 1 Particle Swarm Optimization for Fixture Layout Optimization

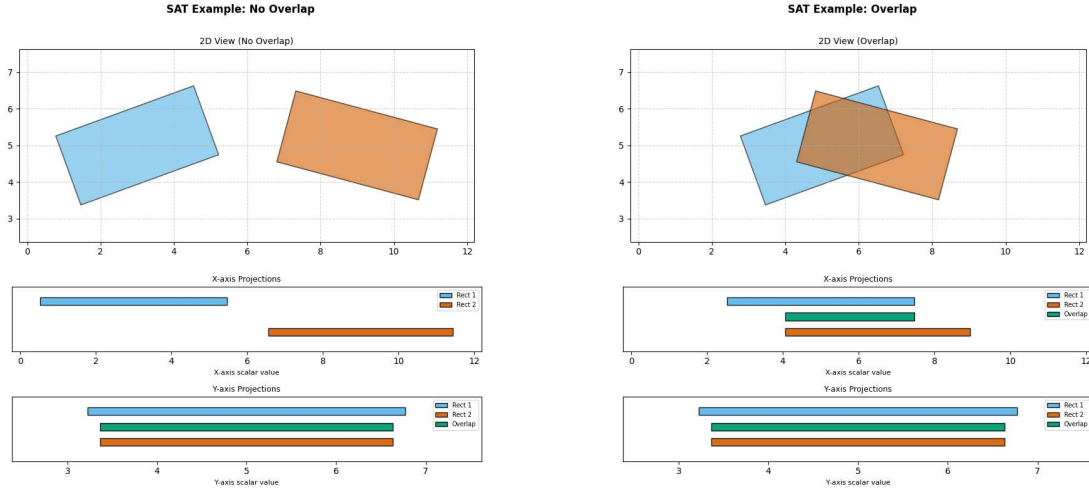
Require: Swarm size `swarm_size`, maximum number of iterations `max_iter`, inertia weight w , cognitive coefficient c_1 , social coefficient c_2 , variable bounds, initial solution P_0 , maximum number of fixtures `NMax`

Ensure: Feasible solution P^* not worse than P_0

```

1: for  $s = 1$  to swarm_size do
2:   if  $s = 1$  then
3:      $P^* \leftarrow P_s^* \leftarrow P_s \leftarrow P_0$ 
4:   else
5:      $P_s^* \leftarrow P_s \leftarrow P_0 + \text{random perturbation}$ 
6:   end if
7:   if  $cv(P_s) = 0$  then
8:     if  $f_{iner}(P_s) > f_{iner}(P^*)$  then
9:        $P^* \leftarrow P_s$ 
10:    end if
11:  end if
12:  Randomly initialize velocity matrix  $V_s$ 
13: end for
14: for  $k = 1$  to max_iter do
15:   for  $s = 1$  to swarm_size do
16:    for  $i = 1$  to 4 do
17:      for  $j = 1$  to NMax do ▷ Update velocity and position
18:        Randomly generate  $r_1, r_2 \in [0, 1]$ 
19:         $V_s[i, j] \leftarrow wV_s[i, j] + c_1r_1(P_s^*[i, j] - P_s[i, j]) + c_2r_2(P^*[i, j] - P_s[i, j])$ 
20:         $P_s[i, j] \leftarrow P_s[i, j] + V_s[i, j]$ 
21:      end for
22:    end for
23:    for  $j = 1$  to NMax do
24:      if fixture  $j$  lies outside the workpiece then
25:        Identify the nearest workpiece vertex and the fixture vertex farthest from it
26:        Translate fixture  $j$  by the minimum displacement required to satisfy the bound-
        ary constraint
27:      end if
28:      if fixture  $j$  has type 0 then
29:        Set  $P_s[1, j] \leftarrow \text{WTab}$  and  $P_s[2, j] \leftarrow \text{HTab}$ 
30:      end if
31:    end for
32:    if  $cv(P_s) = 0$  and  $cv(P_s^*) = 0$  then ▷ Deb's rules
33:      if  $f_{iner}(P_s) > f_{iner}(P_s^*)$  then
34:         $P_s^* \leftarrow P_s$ 
35:      if  $f_{iner}(P_s^*) > f_{iner}(P^*)$  then
36:         $P^* \leftarrow P_s^*$ 
37:      end if
38:    end if
39:    else if  $cv(P_s) < cv(P_s^*)$  then
40:       $P_s^* \leftarrow P_s$ 
41:    end if
42:  end for
43: end for
44: return  $P^*$ 

```



(a) Separation Axis Theorem with no overlap

(b) Separation Axis Theorem with overlap

Figure 7.2: Separation Axis Theorem for oriented rectangles. Each figure consists of three subplots. The first subplot illustrates the spatial configuration of the rectangles. The second and third subplots show the projections of their principal axes onto the x - and y -axes, respectively. An overlap occurs when the projections intersect along both axes; the overlap is highlighted in black in the second and third subplots.

edge-to-edge configurations.

For this reason, the *Separation Axis Theorem* [34] is adopted to enforce the non-overlap constraint. The theorem states that two oriented rectangles do not overlap if and only if there exists at least one axis along which their projections are disjoint. Such an axis is called a *separating axis*. Therefore, non-overlap is verified by projecting both rectangles onto the candidate separating axes and checking whether their projections are disjoint on at least one of them, as illustrated in Figure 7.2.

During the search phase (lines 18–20 of Algorithm 1), the velocity matrix V_s and the position matrix P_s associated with particle s are iteratively updated according to the standard PSO update equations.

Whenever possible, constraint violations arising from updated fixture positions are mitigated through dedicated geometric repair procedures, as reported in lines 24–26 of Algorithm 1. For each fixture, if its geometry lies partially or en-

tirely outside the workpiece boundary, the nearest vertex of the workpiece polygon and the fixture vertex farthest from it are identified. The fixture is then translated by the minimum displacement required to satisfy the boundary constraint. The geometric repair strategy is limited to boundary violations, as these constraints are the most suitable for correction without adversely affecting the satisfaction of other constraints.

Lines 28–30 of Algorithm 1 report another constraints reparation strategy, where fixtures with type zero, representing unused fixtures, are directly assigned with the dummy lower-left corner coordinates ($WTab, HTab$), as in the CP model formulation (see Section 7.2.2), and are excluded from constraint violation evaluations.

To guide the search toward feasible positions, as illustrated in lines 32–41 of Algorithm 1, candidate solutions are evaluated using Deb’s rules [15]:

- Between two feasible solutions, retain the one with the higher objective value.
- Between a feasible and an infeasible solution, retain the feasible one.
- Between two infeasible solutions, retain the one with the lower constraint violation.

After updating the personal best position P_s^* of each particle s using Deb’s rules, the global best position P^* of the swarm is updated accordingly. At the end of the final iteration, the best position found by the swarm is returned.

7.5 Evaluation

We evaluated the proposed resolution strategy on seven distinct workpieces: a stair step of a spiral staircase, a stair step of a simple staircase, a music speaker, a dashboard, a coffee table, a door, and a door with a porthole (see Table 7.3 and Table 7.4).

The work center considered was equipped with 24 square fixtures with dimensions 145×145 mm and 12 rectangular fixtures with dimensions 180×65 mm. The minimum and maximum numbers of fixtures were set to $NMin = 3$ and $NMax = 6$,

respectively, for all workpieces except the coffee table and the two door workpieces, which, due to their larger dimensions, required $N_{Min} = 6$ and $N_{Max} = 10$, and $N_{Min} = 10$ and $N_{Max} = 20$, respectively.

Both the CP and MIP models were implemented in MiniZinc version 2.9.5 [43]. The CP model was solved using the CP solvers Gecode [56], Chuffed [13], and OR-Tools [17], whereas the MIP model was solved using the commercial solver Gurobi [18].

We also considered *Gecode + LNS*, a variant of Gecode implementing Large Neighborhood Search (LNS) [48], to solve the CP model, configuring it to explore the search space by randomly fixing 80% of the decision variables corresponding to the x - and y -coordinates, adopting a constant restart strategy triggered every 1000 explored nodes.

All solvers were executed in single-threaded mode to ensure a fair comparison. The experiments were performed on a 64-bit Windows machine equipped with an Intel Core i5 processor and 16 GB of RAM.

Table 7.1 reports the values obtained for the objective function f_{dist}^{area} for the different workpieces, together with the corresponding *a posteriori* values of f_{iner} , and the solve time t_{solve} required to reach the optimal solution.

Identical values of f_{dist}^{area} do not necessarily correspond to identical values of f_{iner} , as observed for the Dashboard workpiece. This is because the moments of inertia are highly sensitive to little displacement; therefore, if the solutions obtained by the solvers do not result in identical fixture placements, the value of f_{iner} may differ.

Among the CP solvers, Gecode delivers the best overall performance, finding feasible solutions for all the considered workpieces, whereas Chuffed and OR-Tools fail to find feasible solutions for the two door workpieces.

Gecode + LNS achieves the best results for the two door workpieces, producing layouts that outperform those obtained by Gurobi with the MIP model. For the remaining workpieces, however, Gurobi finds solutions that are either superior or equivalent to those obtained by the CP solvers with respect to f_{dist}^{area} . This can be partly attributed to the use of continuous decision variables in the MIP model, which enlarges the feasible region and may therefore yield higher-quality solutions.

If we consider the information of the solve time presented in Table 7.1, among

the CP solvers, with an imposed time limit of 300 s, Chuffed is the most robust, finding the optimal solution in four of the seven instances, and it is the only CP solver able to prove optimality for the stiar step of the simple staircase. Gecode exhibits the greatest variability in performance, ranging from competitive run-times on the easier instances to requiring 165.018 s for the speaker workpiece, while Chuffed and OR-Tools solve the same instance in only 0.106 s and 0.120 s, respectively.

Being a local-search-based approach, *Gecode + LNS* cannot prove optimality. Nevertheless, it reaches the same optimal value of f_{dist}^{area} as the other CP solvers for the simpler workpieces and achieves the best value of f_{iner} in four out of the seven considered workpieces, demonstrating the effectiveness of a local search strategy.

Table 7.1: Objective values for f_{dist} and f_{iner} , and solve time t_{solve} in seconds to reach the best f_{dist} solution. Timeout (300 s) is indicated by “—”. Solutions where the solver reached optimality are marked with *. For each workpiece, the maximum values of f_{dist} and f_{iner} are highlighted in bold, including ties.

Workpiece	Metric	Gecode	Gecode+LNS	Chuffed	OR-Tools	Gurobi
Stair step spiral staircase	f_{dist}	45655*	45655	45655*	45655*	45662.155*
	f_{iner}	2.13968×10^9	2.13968×10^9	2.13960×10^9	2.13968×10^9	2.17089×10^9
	t_{solve}	0.559	—	0.326	0.120	2.930
Stair step simple staircase	f_{dist}	72823	72823	72823*	72721	72826.000*
	f_{iner}	6.76573×10^9	6.76573×10^9	6.76530×10^9	6.48172×10^9	6.76638×10^9
	t_{solve}	—	—	3.386	—	0.220
Dashboard	f_{dist}	55582*	55582	55582*	55582*	55582.000*
	f_{iner}	7.10094×10^9	7.10094×10^9	7.08735×10^9	7.08735×10^9	7.08735×10^9
	t_{solve}	0.044	—	0.057	0.102	0.157
Speaker	f_{dist}	49548*	49548	49548*	49548*	49548.000*
	f_{iner}	2.93433×10^9	2.93433×10^9	2.93433×10^9	2.93433×10^9	2.93433×10^9
	t_{solve}	165.018	—	0.106	0.120	0.126
Coffee table	f_{dist}	137262	185351	192711	122920	201686.505
	f_{iner}	2.20750×10^{10}	2.19967×10^{10}	2.21449×10^{10}	9.10490×10^9	2.19472×10^{10}
	t_{solve}	—	—	—	—	—
Door	f_{dist}	261757	521632	—	—	355744.000
	f_{iner}	1.08442×10^{11}	1.52925×10^{11}	—	—	1.17690×10^{11}
	t_{solve}	—	—	—	—	—
Door with porthole	f_{dist}	261759	393343	—	—	298186.000
	f_{iner}	1.08422×10^{11}	1.52861×10^{11}	—	—	1.39714×10^{11}
	t_{solve}	—	—	—	—	—

7.5. EVALUATION

Table 7.2: Comparison of the best and mean moment of inertia and coefficient of variation (CV) obtained with CP and MIP across repeated PSO runs. CV is the sample standard deviation divided by the mean. Bold values indicate the highest best or mean result for each workpiece.

Workpiece	Model	Best	Mean	CV (%)
Stair step spiral staircase	CP	2.38514×10^9	2.34680×10^9	0.936%
	MIP	2.38520×10^9	2.34714×10^9	0.945%
Stair step simple staircase	CP	6.81046×10^9	5.26939×10^9	7.799%
	MIP	6.81109×10^9	6.81108×10^9	0.0001%
Dashboard	CP	7.28465×10^9	7.28363×10^9	0.011%
	MIP	7.28465×10^9	7.28387×10^9	0.011%
Speaker	CP	3.00956×10^9	3.00092×10^9	0.641%
	MIP	3.00616×10^9	2.96697×10^9	0.773%
Coffee table	CP	2.27774×10^{10}	2.25603×10^{10}	0.594%
	MIP	2.27949×10^{10}	2.22810×10^{10}	1.086%
Door	CP	1.78090×10^{11}	1.75895×10^{11}	1.119%
	MIP	1.50433×10^{11}	1.37290×10^{11}	5.093%
Door with porthole	CP	1.70552×10^{11}	1.70511×10^{11}	0.021%
	MIP	1.58425×10^{11}	1.51954×10^{11}	2.823%

For each workpiece, the best CP and MIP solutions in terms of f_{iner} , were used as initial ones for the PSO algorithm.

To enable direct integration into the target industrial system, the PSO algorithm was implemented in Java 18.0.2. Its hyper-parameters were tuned using the Optuna framework [1], yielding the following configuration: `swarm_size` = 1374, `max_iter` = 2912, $w = 0.669$, $c_1 = 2.385$, and $c_2 = 0.558$.

Table 7.2 reports the best and average results, together with the coefficient of variation (CV), obtained over 20 PSO runs initialized from the CP and MIP solutions. Overall, the two initialization methods show comparable best-case performance, while CP provides better average results across repeated runs, achieving the highest mean value for four workpieces compared with three for MIP.

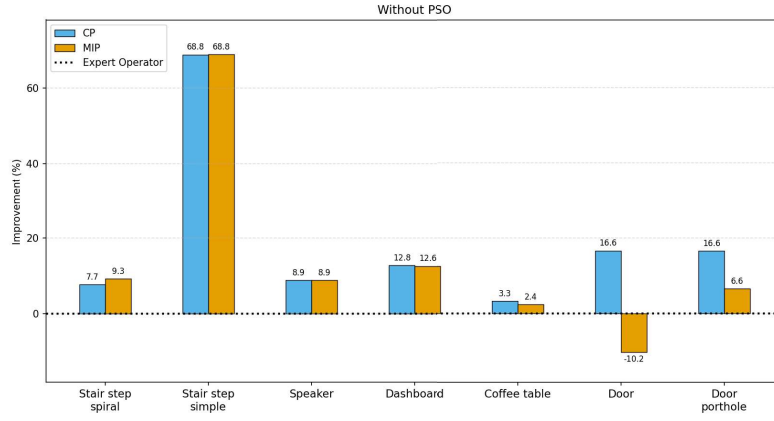
The coefficient of variation further indicates that CP generally provides more consistent results for the more challenging workpieces, particularly for the coffee table and the two door instances.

Figure 7.3 shows the percentage improvement in f_{iner} with respect to the layout

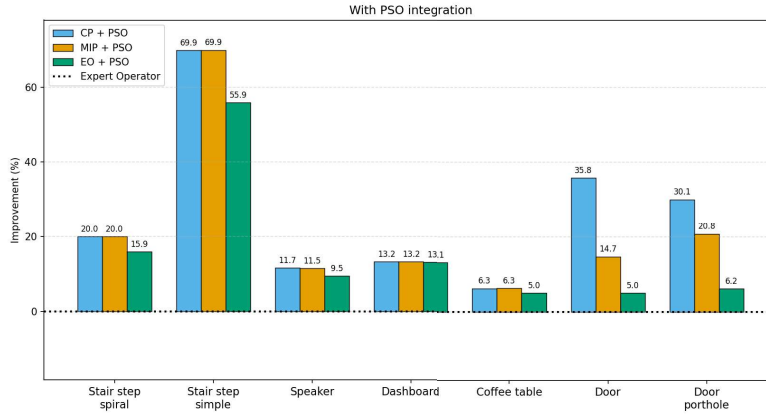
7.5. EVALUATION

proposed by an expert operator. The first plot (Figure 7.3a) reports the improvements achieved by the CP model and the MIP model before PSO refinement, whereas the second plot (Figure 7.3b) shows the corresponding results after PSO refinement, including those obtained by initializing PSO from the expert layout.

The largest workpieces, namely the coffee table, the door, and the door with porthole, represent the most challenging instances. Among the considered ap-



(a) Percentage improvement over expert operator



(b) Percentage improvement over expert operator after PSO refinement

Figure 7.3: Percentage improvement relative to the solution provided by the expert operator. The first plot illustrates the improvements achieved by the CP model and the MIP model without PSO integration, whereas the second plot presents the improvements obtained with PSO integration, alongside the improvement of the expert operator's solution when combined with PSO.

proaches, the CP model consistently achieves the best improvements over the expert layout and is the only one that improves upon the expert solution for the door workpiece.

Among the considered approaches, the CP model consistently achieves the greatest improvements over the expert layouts and is able to outperform the expert solution for the door workpiece, for which the MIP model fails.

The plot in Figure 7.3b reports in green the improvement achieved by initializing PSO with the expert operator layout, and for all the considered workpieces, PSO generates layouts that outperform the expert solution, demonstrating its effectiveness as a refinement technique.

Tables 7.3 and 7.4 report visual comparisons of the best solutions obtained with the CP model and the MIP model for the different workpieces, together with the corresponding solutions refined by PSO. Note that, in the figures, the support bars can be positioned beneath the areas to be extruded, since the fixture height prevents collisions between the cutting tools and the bars.

Rotation plays an important role in the stair step of the spiral staircase and coffee table workpieces, where fixture orientation improves adherence to the workpiece boundary. For the speaker and the dashboard workpiece, the PSO refinement introduce only a slight rotation of the rectangular fixtures, yielding the marginal improvement reported in Figure 7.3.

The Stair step of the simple staircase exhibits the largest improvement over the expert layout. This is mainly due to the use of a larger number of fixtures, five for the CP and MIP models, compared with the three fixtures used by the expert operator.

For both the door and the door with a porthole workpieces, the expert operator adopts a symmetric fixture layout, whereas the PSO refinement tends to move the fixtures farther from the workpiece centre. Since the principal moments of inertia are maximised when the fixtures are placed away from the symmetry axes, this configuration yields higher values of f_{iner} . Moreover, in the door workpiece, the expert layout relies exclusively on square fixtures, whereas the optimization strategies increase the number of fixtures using even rectangular ones, thereby increasing the supported area of the workpiece.

Overall, considering both the solutions obtained by CP and MIP independently

and their subsequent refinement with PSO, the best results are obtained with the CP model. The CP formulation is expected to scale better to problems involving additional fixture types and multiple areas to be extruded owing to the stronger propagation of the `diffn` global constraint and the fact that type and overlap decisions are modeled through discrete variables that cannot be relaxed, unlike the x and y variables in the MIP formulation.

Table 7.3: Summary of the best solutions, corresponding solution providers, and moment of inertia values for the first four workpieces. The workpiece perimeter is shown with solid lines. Dashed lines indicate extrusion areas, fixture regions are marked with diagonal hatching, and supporting bars are shown as shaded rectangles.

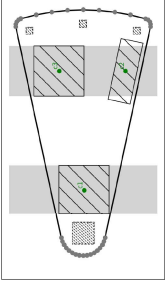
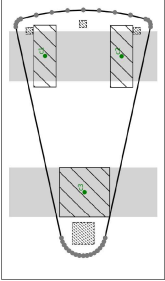
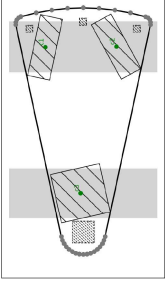
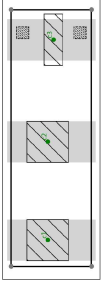
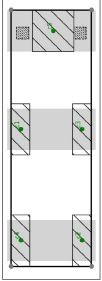
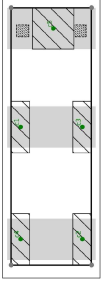
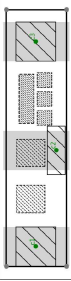
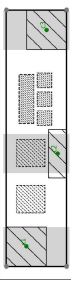
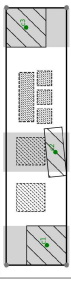
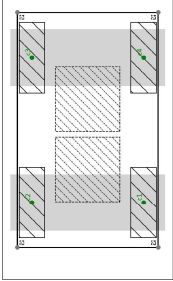
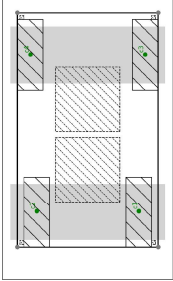
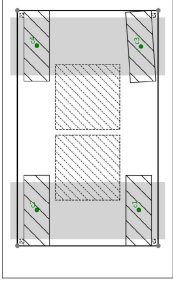
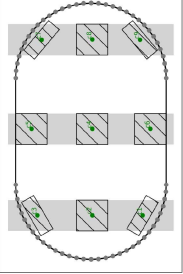
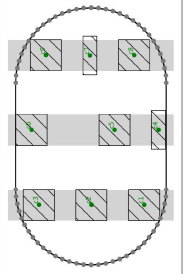
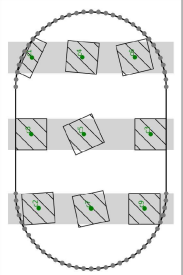
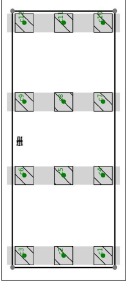
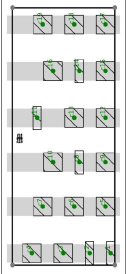
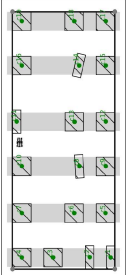
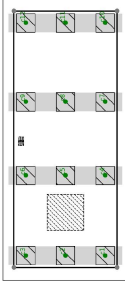
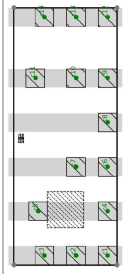
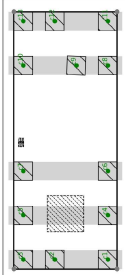
Workpiece	Expert Operator	Best CP/MIP	Best with PSO
Stair step spiral staircase	1.98697×10^9	MIP: 2.17089×10^9	MIP: 2.38520×10^9
			
Stair step simple staircase	4.00911×10^9	MIP: 6.76638×10^9	MIP: 6.81108×10^9
			
Dashboard	6.29544×10^9	CP: 7.10094×10^9	CP: 7.12882×10^9
			
Speaker	2.69490×10^9	CP, MIP: 2.93433×10^9	CP: 3.00956×10^9
			

Table 7.4: Summary of the best solutions, corresponding solution providers, and moment of inertia values for the remaining three workpieces. The workpiece perimeter is shown with solid lines. Dashed lines indicate extrusion areas, fixture regions are marked with diagonal hatching, and supporting bars are shown as shaded rectangles.

Workpiece	Expert Operator	Best CP/MIP	Best with PSO
Coffee table	2.14371×10^{10}	$CP: 2.21449 \times 10^{10}$	$MIP: 2.27948 \times 10^{10}$
			
Door	1.31109×10^{11}	$CP: 1.52925 \times 10^{11}$	$CP: 1.78090 \times 10^{11}$
			
Door with porthole	1.31109×10^{11}	$CP: 1.52861 \times 10^{11}$	$CP: 1.70552 \times 10^{11}$
			

Chapter 8

FlatZinc based Particle Swarm Optimization

In Section 7.4, we stated that the linear constraints of the MIP model were used as a reference to model the constraints in PSO and incorporated into its implementation to evaluate the feasibility of the layouts.

Implementing this approach, however, requires the MIP constraints to be reformulated into a form that can be evaluated by PSO. Since the MIP model was developed in MiniZinc, this reformulation can be avoided by exploiting the FlatZinc file generated during model compilation. The constraint evaluation performed by PSO can then be based directly on the information contained in this file.

This chapter presents the implementation of this solution through the development of a FlatZinc-based PSO algorithm designed to solve MiniZinc models involving continuous variables and nonlinear constraints. Before describing the algorithm in detail, the following sections discuss the structure of the FlatZinc model and how it is represented within the FlatZinc-based PSO.

8.1 FlatZinc model and flattening process

The process of converting a MiniZinc model into a FlatZinc model is known as *flattening*. During this process, the variables of the original model are retained, arrays of variables are expanded into indexed variables, and constraints are rewrit-

ten into a primitive form supported by the target solver. Listing 8.2 illustrates this process by presenting the FlatZinc model generated from the MiniZinc model presented in Listing 8.1.

Listing 8.1: Example of MiniZinc model

```
1  var -1.0..1.0: x1;
2  var -1.0..1.0: x2;
3
4  var float: objective =
5  pow(x1, 2.0) + pow(x2 - 1.0, 2.0);
6
7  constraint x2 - pow(x1, 2.0) = 0.0;
8
9  solve minimize objective;
```

Listing 8.2: JSON representation of the FlatZinc model

```
1  {
2    "variables": {
3      "x1" : { "type" : "float", "domain" : [[-1.0, 1.0]] },
4      "x2" : { "type" : "float", "domain" : [[-1.0, 1.0]], "
        defined" : true },
5      "objective" : { "type" : "float", "defined" : true },
6      "X INTRODUCED_1_" : { "type" : "float", "domain" : [[-2.0, 0
        .0]], "introduced": true, "defined" : true },
7      "X INTRODUCED_2_" : { "type" : "float", "introduced": true,
        "defined" : true }
8    },
9    "arrays": {
10   },
11   "constraints": [
12     { "id" : "float_pow", "args" : ["X INTRODUCED_1_", 2.0, "
        X INTRODUCED_2_"], "defines" : "X INTRODUCED_2_" },
13     { "id" : "float_pow", "args" : ["x1", 2.0, "x2"], "defines" :
        "x2" },
14     { "id" : "float_lin_eq", "args" : [[1.0, 1.0, -1.0], ["x2", "
        X INTRODUCED_2_", "objective"], -0.0], "defines" : "
        objective" },
15     { "id" : "float_lin_eq", "args" : [[1.0, -1.0], ["x2", "
        X INTRODUCED_1_"], 1.0], "defines" : "X INTRODUCED_1_" }
```

```
16     ],  
17     "output": ["x1", "x2"],  
18     "solve": { "method" : "minimize", "objective" : "objective" },  
19     "version": "1.0"  
20 }
```

The flattening process may introduce new variables, whose values may or may not be defined by existing constraints. For instance, for the model in Listing 8.1, the flattening process introduces two new variables, `X INTRODUCED_1_` and `X INTRODUCED_2_`, defined in lines 6–7 of Listing 8.2. These variables represent, respectively, a linear transformation of $x2$ and the result of the operation $(x2 - 1)^2$.

In the FlatZinc model, constraints that determine the values of variables are identified by the `defines` attribute, which specifies the corresponding variable. Identifying these variables is important because their values can be computed from the corresponding constraint rather than determined through the search. For instance, in Listing 8.2, all constraints define a variable.

8.2 Invariant graph

Once the details contained in the FlatZinc model have been extracted, they must be represented in a form suitable for evaluation by PSO. In Constraint-Based Local Search (CBLs), models are typically represented as directed graphs linking variables to the constraints in which they occur. This representation is known as the *invariant graph* [30].

In the invariant graph, defined variables have a single incoming edge corresponding to the constraint that assigns their value, as well as outgoing edges to the constraints in which they are involved. Decision variables, in contrast, have no incoming edges and only outgoing edges associated with the constraints in which they participate.

Figure 8.1 represents the invariant graph of the MiniZinc model presented in Listing 8.1, although the model specifies two variables, $x1$ and $x2$, the flattening process identifies only $x1$ as a decision variable. The variable $x2$ is defined by the `float_pow` constraint, which assigns its value and is therefore connected to $x2$ through an incoming edge.

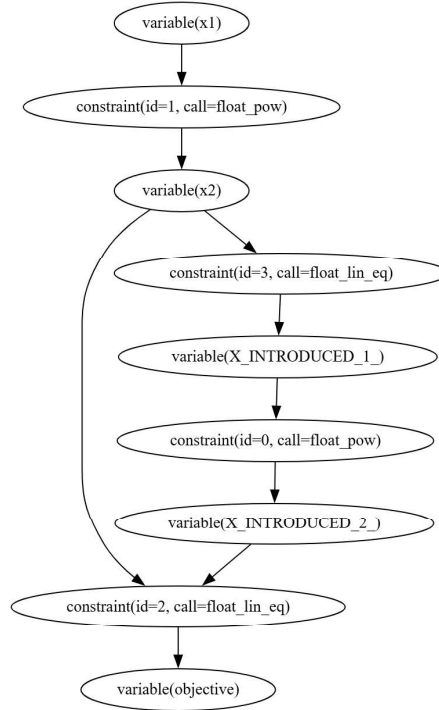


Figure 8.1: Example of invariant graph

The invariant graph allows the values of defined variables to be computed by traversing the graph according to a topological ordering of the constraints. Consequently, when PSO is used to search for a solution, it only needs to assign values to the decision variables that are not defined.

8.3 Integrating FlatZinc and PSO

Solving a constraint optimization problem with PSO using the information contained in the FlatZinc model and soft constraints requires associating each built-in FlatZinc function with a corresponding violation function.

In Flatzinc-based PSO, each violation function measures the distance from the feasible region, as this strategy has been shown to be more effective than death-based penalties [49, 42]. For instance, the constraint `float_pow` is represented in FlatZinc by the built-in function:

```
float_pow(var float:x, var float:y, var float:z)
```

corresponding to the relation $z = x^y$. Its violation can therefore be expressed as:

$$\max(|x^y - z|, 0)$$

The FlatZinc-based PSO must assign a value to each decision variable identified in the invariant graph $\mathcal{G}$ derived from the MiniZinc model. Therefore, the position of each particle $s \in [1..\text{swarm_size}]$ is represented by an array p_s of dimension d , where d is the number of decision variables in $\mathcal{G}$.

Once the particle position p_s has been determined, the values of the defined variables are computed by traversing the constraints in $\mathcal{G}$ according to their topological order, and the violation of each constraint is evaluated using its corresponding violation function.

The overall constraint violation associated with particle p_s is computed as the sum of the violations of all constraints represented in $\mathcal{G}$:

$$cv(p_s) = \sum_{c \in \mathcal{C}} cv_c(p_s),$$

where $\mathcal{C}$ denotes the set of constraints represented in the invariant graph $\mathcal{G}$, and $cv_c(p_s)$ denotes the violation associated with constraint c .

Algorithms 2 and 3 present the pseudocode of the FlatZinc-based PSO. The algorithm takes as input the swarm size `swarm_size`, the maximum number of iterations `max_iter`, the inertia weight w , the cognitive coefficient $c1$, the social coefficient $c2$, and the invariant graph $\mathcal{G}$ of the FlatZinc model. It returns the best solution found by the swarm, denoted by p^* .

At the beginning of the initialization phase in lines 1–5 of Algorithm 2, each particle is initialized from the invariant graph $\mathcal{G}$ by considering the number of decision variables d and their corresponding bounds. The particle position p_s and velocity v_s are randomly initialized, and a stagnation counter k_s is set to zero. The counter tracks the number of consecutive iterations in which the particle fails to improve its personal best. Since strict adherence to Deb’s rules can lead to

Algorithm 2 FlatZinc-Based Particle Swarm Optimization initialization phase

Require: Swarm size `swarm_size`, maximum number of iterations `max_iter`, inertia weight w , cognitive coefficient c_1 , social coefficient c_2 , invariant graph $\mathcal{G}$

Ensure: Best solution found p^*

```

1: Let  $d$  be the number of variables identified in  $\mathcal{G}$ 
2: for  $s = 1$  to swarm_size do
3:   Randomly initialize  $p_s$  within the bounds of the variables in  $\mathcal{G}$ 
4:   Randomly initialize velocity vector  $v_s$ 
5:    $k_s \leftarrow 0$ 
6:   Evaluate  $p_s^*$  computing  $cv(p_s^*)$  and  $f(p_s^*)$ 
7:    $p_s^* \leftarrow p_s$ 
8:   if  $cv(p_s^*) = 0$  then
9:     if  $f(p_s^*) < f(p^*)$  then
10:       $p^* \leftarrow p_s^*$ 
11:    end if
12:  else if  $cv(p_s^*) < cv(p^*)$  then
13:     $p^* \leftarrow p_s^*$ 
14:  end if
15: end for

```

premature convergence, the particle is reinitialized when the counter reaches a predefined threshold, promoting exploration and helping the search escape local optima.

Once the first position is assigned, each particle is evaluated in terms of objective value and constraint violations, as described in line 6 of Algorithm 2. In this phase, the solution proposed by the particle is injected into the invariant graph $\mathcal{G}$ and the values of the defined variables as well as the constraint violations are computed.

At the end of the evaluation (lines 7–14 of Algorithm 2), the personal best position p_s^* of each particle is set to its first evaluated solution, and the global best solution p^* is updated considering the best candidate found among all particles.

During the search phase, in lines 19–20 of Algorithm 3, the velocity of each particle is iteratively update according to the standard PSO equations.

In lines 21–25 of Algorithm 3, uniformly sampled noise $\epsilon \in (-\delta, \delta)$ is added to the particle velocity when a randomly generated value ρ is below 0.5. These perturbations allow particles to occasionally deviate from their current trajectories, helping them escape local optima and promoting broader exploration of the search space [20].

After updating the velocity, the position is updated accordingly, and the parti-

Algorithm 3 FlatZinc-Based Particle Swarm Optimization search phase

```
16: for  $k = 1$  to max_iter do
17:   for  $s = 1$  to swarm_size do
18:     for  $i = 1$  to  $d$  do
19:       Randomly generate  $r_1, r_2 \in [0, 1]$ 
20:        $v_s[i] \leftarrow wv_s[i] + c_1r_1(p_s^*[i] - p_s[i]) + c_2r_2(p^*[i] - p_s[i])$ 
21:       Randomly generate  $\rho \in [0, 1]$ 
22:       if  $\rho < 0.5$  then
23:         Randomly generate  $\epsilon \in [-\delta, \delta]$ 
24:          $v_s[i] \leftarrow v_s[i] + \epsilon$  ▷ Perturbed velocity
25:       end if
26:        $p_s[i] \leftarrow p_s[i] + v_s[i]$ 
27:     end for
28:     Evaluate  $p_s$  and compute  $cv(p_s)$  and  $f(p_s)$ 
29:     if  $k_s = k_{\max}$  then ▷ Stagnation prevention
30:       Reinitialize  $p_s$  and  $v_s$ 
31:        $p_s^* \leftarrow p_s$ 
32:        $k_s \leftarrow 0$ 
33:     end if
34:     if  $cv(p_s) = 0$  and  $cv(p_s^*) = 0$  then
35:       if  $f(p_s) < f(p_s^*)$  then
36:          $p_s^* \leftarrow p_s$ 
37:          $k_s \leftarrow 0$ 
38:         if  $f(p_s^*) < f(p^*)$  then
39:            $p^* \leftarrow p_s^*$ 
40:         end if
41:       end if
42:       else if  $cv(p_s) < cv(p_s^*)$  then
43:          $p_s^* \leftarrow p_s$ 
44:          $k_s \leftarrow 0$ 
45:       else
46:          $k_s \leftarrow k_s + 1$ 
47:       end if
48:     end for
49:   end for
50: return  $p^*$ 
```

cle is evaluated in terms of constraint violation and objective function (lines 26–28 of Algorithm 3).

At the end of the evaluation, the stagnation mechanism is applied. If a particle does not improve its personal best for $k_{\max}$ consecutive iterations, it is reinitialized as described in lines 29–33 of Algorithm 3.

Candidate solutions are compared using Deb’s rules, and after updating the personal best positions, the global best solution p^* is updated accordingly as illustrated in lines 34–47 of Algorithm 3. Finally, at the end of the optimization process, the best solution found is returned.

8.4 Evaluation

To evaluate the effectiveness of the proposed integration, the algorithm described in Section 8.3 was compared with a non-FlatZinc PSO implementation. Both algorithms adopt the same search strategy; the only difference is that the FlatZinc-based PSO derives the variables and constraints from the FlatZinc model, whereas the non-FlatZinc implementation directly encodes the problem variables and constraints in the PSO code.

The two approaches were evaluated on the benchmark problems presented in [20] and [45], comprising a total of 15 problems¹. The experiments were conducted on a 64-bit Windows machine equipped with an Intel Core i5 processor and 16 GB of RAM.

To ensure a fair comparison between the two approaches, the hyperparameters were set to those presented in Section 7.5, which were shown to be effective in guiding the PSO search towards high-quality solutions for constraint optimization problems. Since the benchmark problems are less complex and involve fewer constraints than those considered for fixture layout optimization, only the `swarm_size` and `max_iteration` parameters were reduced, to 100 and 500, respectively.

Table 8.1 presents the results of the comparison between the FlatZinc-based PSO and the non-FlatZinc PSO. The results are evaluated on 50 independent runs

¹Problems 3 and 4 presented in [45] correspond to problems g09 and g04 presented in [20], respectively, whereas problem 5 represents a variant of problem 4 for which the solution is unknown.

8.4. EVALUATION

Table 8.1: Comparison between FlatZinc-based PSO and non-FlatZinc PSO. The best value (closest to the optimum) is highlighted in bold when there is a difference between the values proposed.

Problem	Optimum	Best fzn-PSO	Best PSO	Mean fzn-PSO	Mean PSO
g01	-15.000	-14.892	-14.825	-13.025	-12.887
g02	-0.804	-0.345	-0.341	-0.281	-0.298
g03	-1.000	-0.956	-0.947	-0.895	-0.785
g04	-30665.386	-30666.331	-30666.226	-30665.388	-30664.476
g05	5126.496	–	–	–	–
g06	-6961.813	-6951.099	-6953.812	-6922.202	-6904.423
g07	24.306	32.896	33.607	71.290	47.094
g08	0.095	-0.096	-0.096	-0.096	-0.096
g09	680.630	684.614	685.152	687.721	689.371
g10	7049.248	8015.163	8283.200	12657.795	10026.341
g11	0.750	0.750	0.749	0.750	0.749
g12	-1.000	-1.000	-1.000	-1.000	-1.000
problem 1	1.393	1.999	1.392	1.999	1.392
problem 2	-6961.813	-6946.519	-6953.812	-6922.013	-6904.063
problem 6	-213.000	-211.501	-211.501	-211.501	-211.501

for each problem, using the same random seed for both algorithms to ensure identical initial conditions. Results for problem g05 are omitted, as neither algorithm was able to find a feasible solution.

Considering the results reported in the table for the mean, the FlatZinc-based PSO achieves better performance in 7 of the 15 problems, while the two approaches obtain identical results in 3 problems. Among the remaining cases, excluding g05, the non-FlatZinc PSO performs better in 4 problems.

Since both algorithms share the same search mechanism, the observed performance differences can primarily be attributed to the information provided by the FlatZinc model. As described in Section 8.1, the flattening process produces a more explicit representation of the original model, decomposing complex constraints and identifying defined variables. This additional information allows constraint violations to be evaluated in greater detail and can reduce the number of variables

Table 8.2: Runtime comparison between FlatZinc-PSO and non-FlatZinc PSO.

Problem	fzn-PSO (ms)	PSO (ms)
g01	2260.000	1500.000
g02	6930.000	2510.000
g03	2130.000	1210.000
g04	1660.000	626.760
g05	–	–
g06	984.960	265.970
g07	3040.000	1220.000
g08	878.160	266.950
g09	1900.000	864.160
g10	1670.000	934.520
g11	419.950	266.350
g12	1550.000	726.060
problem 1	702.450	153.410
problem 2	874.010	265.710
problem 6	1210.000	698.410

that must be directly searched.

However, the processing of this additional information comes at the cost of more computational overhead. Table 8.2 reports the runtimes, in milliseconds, required by the FlatZinc-based PSO and the non-FlatZinc PSO to complete 500 iterations. On average, the non-FlatZinc PSO is approximately 2.55 times faster than the FlatZinc-based PSO.

To assess whether the observed differences in solution quality are statistically significant, a *Wilcoxon signed-rank test* [79] was performed on the results of the 50 runs for each benchmark problem. This test compares two related samples by ranking the absolute differences between paired observations and assessing whether these differences are systematically positive or negative. The resulting $pvalue$ measures the evidence against the null hypothesis that the two methods produce equivalent results; a value of $pvalue < 0.05$ is considered to indicate a statistically significant difference between the two methods.

8.4. EVALUATION

Table 8.3: Wilcoxon signed-rank test p-values comparing FlatZinc PSO and non-Flatzinc PSO across benchmark problems. Values with $p_{value} < 0.05$ are highlighted in bold.

Problem	Wilcoxon p_{value}
g01	0.1781
g02	0.001191
g03	2.994e-11
g04	1.867e-07
g05	–
g06	0.003205
g07	0.2115
g08	6.825e-05
g09	0.007844
g10	0.0009504
g11	1.776e-15
g12	8.882e-15
problem 1	1.776e-15
problem 2	0.01026
problem 6	0.005942

Table 8.3 reports the corresponding p_{value} obtained from the run-by-run comparison of the FlatZinc-based PSO and the non-FlatZinc PSO for the benchmark problems. Overall, statistically significant differences were found for 12 of the 15 benchmark problems considered, indicating that the two approaches produce significantly different solution qualities in the majority of the cases.

Chapter 9

Conclusion

This first part of the thesis focused on the development of an Intelligent Decision Support System (IDSS) for the fixture layout optimization problem in wood industry.

Several solution strategies were developed and evaluated. Initially, a simplified version of the problem was considered, which did not account for the presence of areas to be extruded within the workpiece and for the rotation of the fixtures. This problem was formulated as a CP model implemented in MiniZinc and evaluated using two alternative objective functions. The first, f_{iner} , accounts for the principal moments of inertia of the fixture system, whereas the second, $f_{dist}^{semi-perimeter}$, provides an alternative formulation that approximates f_{iner} by considering the distance and size of the fixtures.

The analysis conducted on a case study showed that f_{iner} is more suitable for identifying optimal fixture layouts, but its computational cost increases substantially with problem complexity. In contrast, $f_{dist}^{semi-perimeter}$ provides good-quality solutions in shorter computation times and is less affected by increasing problem complexity. These results therefore motivated the use of an alternative objective function that approximates the results produced by f_{iner} for exact optimization techniques such as CP or MIP.

The complete problem was subsequently considered, incorporating both the areas to be extruded and the rotation of the fixtures. The CP model was extended accordingly and compared with a newly formulated MIP model. Testing the two

approaches on real-world case studies, the CP model proved to be the most effective solution, with its effectiveness demonstrated by its ability to obtain optimal solutions for the simpler instances when solved with Chuffed, while the use of *Gecode* + *LNS* enabled it to handle the more complex cases. The MIP model solved with Gurobi produced solutions comparable to those of CP for simpler instances, but its performance deteriorated as the problem complexity increased.

None of these approaches, however, explicitly considered fixture rotation during the optimization process, as including arbitrary rotations directly in the decision variables would substantially increase the search space. Rotation was therefore introduced in a subsequent refinement phase based on the PSO meta-heuristic. Starting from the solutions generated by the three approaches, PSO improved the initial layouts in all test cases and also outperformed the layouts provided by an expert operator, with the exception of one case in which the initial solution was of low quality. These results demonstrate the effectiveness of using local search as a refinement mechanism and highlight the advantage of integrating the approaches, as the initial feasible layouts provide PSO with a suitable starting point without requiring it to search for feasibility from scratch.

The application of PSO to the fixture layout optimization problem relied on the linear constraints of the MIP model, implemented in MiniZinc. However, incorporating these constraints into PSO required them to be explicitly reformulated and integrated in its code. To overcome this limitation and enable PSO to solve MiniZinc models involving continuous variables and nonlinear constraints, a FlatZinc-based PSO algorithm was developed. The algorithm exploits the information contained in the FlatZinc model, obtained from the compilation of the MiniZinc model, and represents its information through an invariant graph, allowing constraint violations and the values of defined variables to be evaluated directly from this representation.

The comparison with a PSO implementation in which the variables and constraints are directly encoded in the meta-heuristic showed that the FlatZinc-based approach can achieve better solution quality while using the same search mechanism. This improvement can therefore be attributed to the additional information provided by the FlatZinc model, which offers a more explicit description of the problem and can be exploited to guide the search.

Overall, the results demonstrate that complex constrained problems arising in industrial applications often benefit from the integration of exact optimization and local search techniques. The results obtained for the fixture layout optimization problem demonstrate the effectiveness of this integration, while the development of the FlatZinc-based PSO extends this approach beyond the specific application considered and opens the possibility of applying PSO to a broader class of nonlinear models formulated in MiniZinc.
